RISK NETWORK

Spree	**5.6.1**
Stack	**Rails 8.1.3 · Ruby 4.0.1 · PostgreSQL 18**
Date	**13 September 2026**

Written 13 September 2026, in response to the Merchant Risk Network document of 12 September.

That document is good. It absorbed the 11 September findings correctly, it anticipates several problems on its own, and its core principle — the network identifies documented relationships and a human decides — is the right one. This is not a review of it. It is the four things that came out of measuring, each of which changes something in the plan.

One of them corrects a mistake of ours from 11 September.

In one page

1. The strongest identifier does not cross the shops	Card fingerprints are scoped per Stripe account and the five shops are on five different Stripe accounts. Measured.
2. The key custody problem has a name and an answer	A shared keyed hash makes the whole network offline-enumerable. RFC 9497 OPRF is the specific scheme the document leaves open.
3. The IP field is empty, not wrong	Corrects our own 11 September finding. Nothing writes it on the headless path; the CDN addresses we reported were historical.
4. The fulfilment hold has to be built	Spree's approval concept gates nothing in shipping. But `shipment.pend!` exists, so it is wiring rather than a new state machine.

Consequence, in one sentence: **the crypto problem arrives at M4, not M5**, because M4 cannot use the one identifier that would have been safe without it.

---

1. The card fingerprint does not cross his own shops

The document treats the payment instrument identifier as available where a provider exposes one, and correctly says to interpret it by dispute reason. The problem is upstream of that.

Measured on the five retail shops, from the live payment method configuration:

```
shop A   pk_live_51Sz6JADKf…
shop B   pk_live_51Sz7m5Goy…
shop C   pk_live_51Sz7SACh5…
shop D   pk_live_51RYNgnERV…
shop E   pk_live_51Sz8Az2eT…
```

A Stripe publishable key embeds the account. These are **five different Stripe accounts**, not five keys on one account.

And Stripe's own support documentation is explicit that a fingerprint is unique for a card *within a Stripe account*, and that fingerprints for the same card **intentionally differ across accounts**.

So the same card, used at two of his shops, produces two unrelated fingerprints.

Why this matters more than it first looks

The card fingerprint is the only identifier in the whole design that is both **high entropy** and **strongly tied to the person paying**. Email, phone and address are all low entropy and guessable; that is what creates the key custody problem in section 2. The fingerprint is the one field that could have been put behind a plain keyed hash safely, because nobody can enumerate the space of card fingerprints.

Losing it at the cross-shop boundary means M4, the private pilot across his own shops, has to match on exactly the identifiers that are hardest to protect.

What can be done about it

Three options, and the choice is his:

- **Consolidate the shops onto one Stripe account.** Then fingerprints match across all five and M4 becomes both stronger and easier. This is a commercial and operational decision far outside this project, with implications for payouts and reporting, so it is raised rather than recommended.
- **Accept that M4 matches on email, phone and address only**, and treat the fingerprint as a within-shop identifier. This is honest and still useful: a repeat offender at the same shop is caught by it.
- **Ask Stripe** whether a Connect platform arrangement would give a shared fingerprint view. The search result above notes that in a platform and connected account setup the platform can see its own fingerprint across connected accounts, which suggests the question is worth asking rather than assuming.

---

2. Key custody: the document's open question has a specific answer

The document says, correctly:

> A simple SHA-256 hash of an email or phone number is not sufficient because common identifiers can be guessed and hashed offline. The design should use a keyed construction such as HMAC, or another privacy-preserving matching scheme chosen after security review.

The first half is right. The second half is the whole problem, because **HMAC cannot deliver the guarantee either**, and the reason is key custody rather than the algorithm.

For two merchants to match the same person, they must derive the same token from the same identifier. So either every merchant holds the key, or the service does. There is no third arrangement.

Merchants hold the key. Any one participant, or anyone who breaches the weakest of them, can compute the token for any identifier offline. Australian mobile numbers are `04` plus eight digits, about 10^8 candidates; HMAC-SHA256 over a short input runs in the order of 10^7 per second per core, so the entire space falls in seconds on a laptop. Email needs no enumeration at all, because a breach corpus turns it into a list comparison. "We store only tokens" then means "we store a reversible index of every flagged person".

The service holds the key. Then the merchant must send the raw email and phone to be tokenised, at both write and query time. The document's promise that raw customer data stays with the merchant is false at precisely the point that matters, and the service's query log becomes a larger database than the risk list it was built to avoid.

The scheme the security review would arrive at

An Oblivious Pseudorandom Function, standardised in **RFC 9497**. The merchant blinds the identifier, the service evaluates it under its key, the merchant unblinds the result. From the RFC:

- the client learns `F(k, x)`
- the server learns **nothing** about `x`; the RFC calls this *unconditional input secrecy*, and it holds even against an unbounded server
- the client learns nothing about `k`, so cannot precompute the space
- evaluation is **inherently interactive**; there is no offline mode

That removes both horns at once. No merchant can enumerate, because no merchant has the key. No raw identifier reaches the service, because it only ever sees a blinded value. And because every tokenisation is a request, every one of them is rate-limitable and auditable — which turns the document's anti-people-search controls from policy into mechanism.

RFC 9497 defines three modes. The verifiable one, VOPRF, lets the merchant confirm the service used the key it committed to, which matters when the service is a commercial party the merchants are trusting.

Two honest limits

OPRF does not stop the key holder. The service holds `k` and can brute-force its own token store for a low-entropy identifier. It protects merchants from each other and protects raw data in transit; it does not make the operator blind. For a network he operates and profits from, that distinction belongs in the privacy notice rather than being papered over.

It costs availability. Tokenisation becomes a network round trip. That is fine for the risk path, which the document already keeps out of the checkout, but it means a token cannot be computed while the service is down.

When it is actually needed

Not at M4. In the private pilot every shop is his, so he is every party and there is nothing to protect merchants from each other. **Build M4 with no token scheme at all** — match on normalised identifiers directly, because a shared database between shops he owns is an internal design question, not a privacy boundary.

The moment the first merchant he does not control joins, the scheme has to be in place *before* their data lands, because tokens minted under the wrong scheme cannot be retrofitted. That is M5, and the document currently places the security review there too, which is right.

---

3. Correcting ourselves: the IP field is empty, not wrong

On 11 September we reported that `last_ip_address` was recording the CDN rather than the shopper, and the document has carried that forward as a design consequence. The finding was measured across *all* orders, and that was the wrong measurement.

Split by the changeover date instead:

shop	before changeover	after changeover
A	382 orders, **382** with an IP	18 orders, **1** with an IP
B	171 orders, **171** with an IP	10 orders, **0** with an IP

The single exception is dated changeover day itself and carries a CDN address, so it was placed on the old storefront before the switch completed.

So every CDN address we reported was **historical**, written by the old Rails storefront. On the headless path nothing writes the field at all. A full column is not a written column; it can be full because of the past.

What changes. It is not a wrong value to correct, it is a path to create: the storefront must pass the shopper's address on the API call that creates the order, and Spree must record it. The web server has already been corrected to see the real address rather than the edge — that was necessary and it is done — but it is the precondition, not the repair.

The conclusion for the risk list is unchanged and now blunter. IP cannot be a matching identifier, because there is nothing in the field.

---

4. The fulfilment hold is wiring, not a new mechanism

The document's M2 assumes Spree's existing approval concepts can be reused to stop fulfilment. Measured:

```
Spree::Order   approve!, approved?, approver, can_approve?, approvals …
Spree::Shipment  states: pending, ready, canceled, shipped
                 events: ready, pend, ship, cancel, resume
```

Nothing in `shipment.rb` or the stock models references approval at all. So approval is bookkeeping on the order and gates nothing in shipping: an unapproved order can be shipped exactly like any other.

The good news is in the second line. `pend` is a real shipment event, so a hold is `shipment.pend!` plus something to stop it being made ready again. That is wiring an existing mechanism, not building a state machine, and it is a smaller piece of work than "approval does not work" would suggest.

This matters because M2 is the milestone with the money in it. A Risk Alert that does not stop the parcel is a notification, and the loss has already happened.

What we would do first, if asked

Unchanged from 11 September in shape, sharpened by the above:

1. **Capture the dispute.** Still the first thing. Nothing in the stack knows a dispute happened, so there is no history to start from and no deadline alarm. Stripe's dispute object carries the reason and `evidence_details.due_by`.
2. **Wire the hold**, using `shipment.pend!`, because M2 is where the money is.
3. **Build M4 on normalised identifiers with no token scheme**, and accept that the card fingerprint does not cross accounts.
4. **Leave the crypto until a merchant he does not control is joining**, and then use RFC 9497 rather than HMAC.

What was not verified

Stated so nothing here is taken for more than it is. This document came partly out of a research run whose verification stage was cut short by a service limit, so several of its findings were checked by hand afterwards and several were not.

Verified by measurement or primary source: the five Stripe accounts; the account scoping of fingerprints; the OPRF properties; the IP correction; the approval and shipment states.

Not verified, and deliberately left out of the sections above: claims about what competing commercial services do; claims about whether Stripe Radar already covers part of M2; and any estimate of how a regulator would treat the network. Those need their own work before they are repeated to anyone.